

1. CPU utilization should be 100% because for the two processes we defined as part of the `-l` flag we gave 100 as the value after the colon, which means a 100 percent chance to use only the CPU for each instruction. When I use the `-c` and `-p` flags I can see that this is the case.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 5:100,5:100 -c -p
```

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:cpu	READY	1	
3	RUN:cpu	READY	1	
4	RUN:cpu	READY	1	
5	RUN:cpu	READY	1	
6	DONE	RUN:cpu	1	
7	DONE	RUN:cpu	1	
8	DONE	RUN:cpu	1	
9	DONE	RUN:cpu	1	
10	DONE	RUN:cpu	1	

```
Stats: Total Time 10
```

```
Stats: CPU Busy 10 (100.00%)
```

```
Stats: IO Busy 0 (0.00%)
```

2. It is easy to determine that p0 will only take 4 units of time since it performs 4 instructions. From the README I can see that IO processes take a total of 7 units of time: one IO run step and one IO done step, along with 5 blocking units of time. $4 + 7 = 11$ total. We can see that the summary agrees with this assessment.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 4:100,1:0 -c -p
```

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:cpu	READY	1	
3	RUN:cpu	READY	1	
4	RUN:cpu	READY	1	
5	DONE	RUN:io	1	
6	DONE	BLOCKED		1
7	DONE	BLOCKED		1
8	DONE	BLOCKED		1
9	DONE	BLOCKED		1
10	DONE	BLOCKED		1
11*	DONE	RUN:io_done	1	

```
Stats: Total Time 11
```

```
Stats: CPU Busy 6 (54.55%)
```

```
Stats: IO Busy 5 (45.45%)
```

3. The order changes things quite significantly. Since the simulator switches on IO by default, running the blocking IO operation first allows the CPU to be used while IO waits to be done. This reduces the overall time to 7.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 1:0,4:100 -c -p
```

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:io	READY	1	
2	BLOCKED	RUN:cpu	1	1
3	BLOCKED	RUN:cpu	1	1
4	BLOCKED	RUN:cpu	1	1
5	BLOCKED	RUN:cpu	1	1
6	BLOCKED	DONE		1
7*	RUN:io_done	DONE	1	

```
Stats: Total Time 7
```

```
Stats: CPU Busy 6 (85.71%)
```

```
Stats: IO Busy 5 (71.43%)
```

- When we run the program with SWITCH_ON_END it does exactly what it says it will do. It does not switch over to the other process until the first process finishes.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 1:0,4:100 -c -p -S SWITCH_ON_END
```

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:io	READY	1	
2	BLOCKED	READY		1
3	BLOCKED	READY		1
4	BLOCKED	READY		1
5	BLOCKED	READY		1
6	BLOCKED	READY		1
7*	RUN:io_done	READY	1	
8	DONE	RUN:cpu	1	
9	DONE	RUN:cpu	1	
10	DONE	RUN:cpu	1	
11	DONE	RUN:cpu	1	

```
Stats: Total Time 11
Stats: CPU Busy 6 (54.55%)
Stats: IO Busy 5 (45.45%)
```

- When we set the switching behavior to switch on IO the program will run the second process while the first is blocked.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 1:0,4:100 -c -p -S SWITCH_ON_IO
```

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:io	READY	1	
2	BLOCKED	RUN:cpu	1	1
3	BLOCKED	RUN:cpu	1	1
4	BLOCKED	RUN:cpu	1	1
5	BLOCKED	RUN:cpu	1	1
6	BLOCKED	DONE		1
7*	RUN:io_done	DONE	1	

```
Stats: Total Time 7
Stats: CPU Busy 6 (85.71%)
Stats: IO Busy 5 (71.43%)
```

- System resources are not being used effectively this way. When we combine running IO later with switching on IO we end up leaving the CPU idle while p0 runs its remaining IOs, since those IOs do not start until every other process has already finished. Our goal should be to run IO early and often to fully utilize the IO blocking windows to run CPU for other processes.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -c -p -I IO_F
```

Time	PID: 0	PID: 1	PID: 2	PID: 3	CPU	IOs
1	RUN:io	READY	READY	READY	1	
2	BLOCKED	RUN:cpu	READY	READY	1	1
3	BLOCKED	RUN:cpu	READY	READY	1	1
4	BLOCKED	RUN:cpu	READY	READY	1	1
5	BLOCKED	RUN:cpu	READY	READY	1	1
6	BLOCKED	RUN:cpu	READY	READY	1	1
7*	READY	DONE	RUN:cpu	READY	1	
8	READY	DONE	RUN:cpu	READY	1	
9	READY	DONE	RUN:cpu	READY	1	
10	READY	DONE	RUN:cpu	READY	1	
11	READY	DONE	RUN:cpu	READY	1	
12	READY	DONE	DONE	RUN:cpu	1	
13	READY	DONE	DONE	RUN:cpu	1	
14	READY	DONE	DONE	RUN:cpu	1	
15	READY	DONE	DONE	RUN:cpu	1	
16	READY	DONE	DONE	RUN:cpu	1	
17	RUN:io_done	DONE	DONE	DONE	1	

18	RUN:io	DONE	DONE	DONE	1	
19	BLOCKED	DONE	DONE	DONE		1
20	BLOCKED	DONE	DONE	DONE		1
21	BLOCKED	DONE	DONE	DONE		1
22	BLOCKED	DONE	DONE	DONE		1
23	BLOCKED	DONE	DONE	DONE		1
24*	RUN:io_done	DONE	DONE	DONE	1	
25	RUN:io	DONE	DONE	DONE	1	
26	BLOCKED	DONE	DONE	DONE		1
27	BLOCKED	DONE	DONE	DONE		1
28	BLOCKED	DONE	DONE	DONE		1
29	BLOCKED	DONE	DONE	DONE		1
30	BLOCKED	DONE	DONE	DONE		1
31*	RUN:io_done	DONE	DONE	DONE	1	

Stats: Total Time 31

Stats: CPU Busy 21 (67.74%)

Stats: IO Busy 15 (48.39%)

- When we run IO immediately we see much better results because by running IO early and often we can maximize our overlap between CPU and blocking operations. Running IO back to back runs the longest operations efficiently by ensuring we can fully utilize all blocked slots to run CPU for the other processes.

```
ubuntu@cs5600:~/ostep-homework/cpu-intro$ python3 process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -c -p -I IO_RUN_IMMEDIATE
```

Time	PID: 0	PID: 1	PID: 2	PID: 3	CPU	IOs
1	RUN:io	READY	READY	READY	1	
2	BLOCKED	RUN:cpu	READY	READY	1	1
3	BLOCKED	RUN:cpu	READY	READY	1	1
4	BLOCKED	RUN:cpu	READY	READY	1	1
5	BLOCKED	RUN:cpu	READY	READY	1	1
6	BLOCKED	RUN:cpu	READY	READY	1	1
7*	RUN:io_done	DONE	READY	READY	1	
8	RUN:io	DONE	READY	READY	1	
9	BLOCKED	DONE	RUN:cpu	READY	1	1
10	BLOCKED	DONE	RUN:cpu	READY	1	1
11	BLOCKED	DONE	RUN:cpu	READY	1	1
12	BLOCKED	DONE	RUN:cpu	READY	1	1
13	BLOCKED	DONE	RUN:cpu	READY	1	1
14*	RUN:io_done	DONE	DONE	READY	1	
15	RUN:io	DONE	DONE	READY	1	
16	BLOCKED	DONE	DONE	RUN:cpu	1	1
17	BLOCKED	DONE	DONE	RUN:cpu	1	1
18	BLOCKED	DONE	DONE	RUN:cpu	1	1
19	BLOCKED	DONE	DONE	RUN:cpu	1	1
20	BLOCKED	DONE	DONE	RUN:cpu	1	1
21*	RUN:io_done	DONE	DONE	DONE	1	

Stats: Total Time 21

Stats: CPU Busy 21 (100.00%)

Stats: IO Busy 15 (71.43%)

- When using different flags the trace is completely deterministic. In all of the tests I ran IO_RUN_IMMEDIATE allowed the CPU to be used more efficiently than running with IO_RUN_LATER. When IO is run immediately the CPU is able to be used more effectively. In the theoretical worst case you would run all CPU first sequentially, then run all blocking IO; running IO immediately prevents this. SWITCH_ON_IO was the other optimal choice. Without this flag, the CPU time goes to waste when another process could have used it (SWITCH_ON_END). In summary, running IO immediately and freeing the CPU during leads to better CPU utilization in this simulator. These two flags work together to efficiently use the CPU. SWITCH_ON_IO frees the CPU during a block and

`IO_RUN_IMMEDIATE` ensures that the CPU is freed early enough for work to be placed within this gap.